

Personal VLSI/IC-Design Technical Handbook

Digitized from handwritten technical reference notes.

1. Semi-Custom IC Design Flow Overview

1.1 High-Level Flow Sequence

1. **Specification**
2. **RTL Coding**
3. **Functional Simulation**
4. **Synthesis**
5. **Timing Simulation**
6. **Physical Design (PD) / Layout**
 - Floor planning
 - Power planning
 - Placement
 - CTS (Clock Tree Synthesis)
 - Routing
7. **GDS II Generation**

1.2 Key Terminology & Definitions

- **Tcl:** Tool Command Language
- **Synthesis:** The process combining Translation, Logic optimization, and Gate mapping. RTL code and library files (`.lib` , gates) are converted into hardware circuits.
- **Compilation:** Structure check primarily looking for syntactical errors.
- **Elaboration:** Module instantiation and Input/Output (I/O) checking.
- **Simulation:** Behavioral verification of the design.
- **Clk Skew:** The difference in arrival time of a clock signal between two registers.
- **GDS:** Graphical Data Stream Information Interchange.

Additional technical note:

The industry-standard expansion for GDS is "Graphic Design System," though transcribed here exactly as written in the source notes.

2. Front-End Design & Simulation (SimVision / Xcelium)

2.1 File Creation and Compilation

- Open a file editor: `gedit "file name"`
- Launch simulation environment: `nclaunch -new`
- Create `cds.lib` file → Save → Select library file
- **Compile:** Select **Tools** → **Verilog compiler** → **OK**

2.2 Elaboration and Simulation

- **Elaborate:** Go to **worklib** → click **"+"** → select file → **Tools** → **Elaborator** → **OK**
- **Simulate:** Go to **Snapshots** → select `_tb` module → **Tools** → **Simulator**
- **Command Line Execution (Xcelium):** Run the following command in terminal:
`xrun "*.v" "tb_*.v" -access +rwc -gui`

3. Logic Synthesis (Cadence Genus)

3.1 Synthesis Inputs & Outputs

- **Inputs to Genus:** RTL code (`.v` , `.vhd`), Constraints (`.sdc`), and Libraries (`.lib` , `.lef`)
- **Outputs from Genus:** Gate-level Netlist (`.v`), Constraints (`.sdc`), Log files, and Reports
- **Internal Steps:** Generic translation, technology mapping, and power optimization

3.2 Synthesis Execution Flow

1. Create a new folder named `Synthesis` and place the top-level design file inside.
2. Ensure the `.sdc` constraint file (for Clock and Static Timing Analysis) and `.lib` files (gate models) are present.
3. Create a Tcl script file (`script_file.tcl`) to execute multiple commands at once.
4. Open Cadence tools suite and launch `genus` .
5. Source the Tcl script: `source script_file.tcl` (A window will open).
6. **GUI Navigation:** Go to **layout (+)** → **Schematic**.
7. **Timing Debug:** Navigate to **Timing** → **Debug timing** → **OK**.

4. Logical Equivalence Check (Cadence Conformal LEC)

This step verifies that the synthesized netlist functionally matches the original RTL design after Genus synthesis.

4.1 Preparation and Initialization

- Create a `lec` folder.
- Copy the synthesized `netlist.v` and the original RTL `vm.v` from the `Synthesis` folder to the `lec` folder.
- Open a terminal and launch the tool: `lec -auto`

4.2 LEC Setup and Comparison Sequence

1. Set the tool mode to Setup: **Inside tool** → **Current mode** → **setup**
2. Read the standard cell library: **Bottom** → **Setup > read library /home/install/FOUNDRY/digital/90nm/dig/vlog/slow.v**
(Note: Path is PDK/Foundry dependent)
3. Read the original RTL design as golden: **Setup > read design vm.v -golden**
4. Read the synthesized netlist as revised: **Setup > read design vm_netlist.v -revised**
5. Change the tool's current mode from setup to LEC: **Change current mode to** → **lec**
6. Execute comparison: Click **Compare** → **compare** → **compare**
7. Verify mapping: Open **Tools** → **Mapping Manager**
 - If there are no unmapped points, the netlist was generated successfully.
 - For mapped points: Right-click any option → select **Source code** to see the point in both the golden and revised models.
8. *Front-end design is now completed.*
9. Optional Tempus step: **[UNCLEAR: Timing/Incisive] Debug window** → **Tempus** → **tools**

5. Physical Design Flow (Cadence Innovus)

5.1 Initialization and File Import

- Create a separate folder named `pd` for the physical design tools.
- Copy `netlist.v` and the output constraints (`.sdc`) from the Synthesis folder to the `pd` folder.
- Open terminal → launch Cadence tool: `innovus`
- **Import Netlist:** **File** → **Import Design** → **Netlist** → click "three dots" → select `vm_netlist.v` (Auto Assign)
- **Import Technology (LEF):** Navigate to **Technology** → **LEF** (Library Exchange Format) → click "three dots" → browse to `/home/install/FOUNDRY/digital/90nm/dig/lef/`
 - Select `gsclib090_translated.lef` (Note: This specifies a 90nm Generic Standard Cell library, which is PDK dependent).
 - If a memory block is used, include the macro and tech LEF files (e.g., `gsclib090_translated_mac.lef`).

5.2 Multi-Mode Multi-Corner (MMMC) Setup

1. Define global power and ground: Set Power Nets to **VDD** and Gnd Nets to **VSS**.
2. Navigate to **Analysis config** → **Create Analysis config** to open the Multi Mode Multi Corner Browser.
3. **Library Sets:**
 - Right-click **Library Sets** → **New** → Name it **max_lib** → under Timing select **.../dig/lib/slow.lib** → **OK**.
 - Create another set named **min_lib** → under Timing select **.../fast.lib** → **OK**.
4. **RC Corners:**
 - Name = **RC**, Temp = **125**.
 - Set **captable** path (**/.../captable**) and **QRC** path (**/.../QRC**).
5. **Delay Corners:**
 - Create **max_delay** → assign RC corner = **RC**, lib set = **max_lib**.
 - Repeat the same process for **min_delay**.
6. **Constraint Mode:** Name = **constraints_sdc**, SDC constraint = **vm_output_const.sdc**.
7. **Analysis View:**
 - Name = **wc** (worst corner), mapped to **max_delay**.
 - Name = **bc** (best corner), mapped to **min_delay**.
8. **Set Analysis Views:** Setup Analysis view → **wc**, Hold Analysis view → **bc**.
9. Save and close → **Default view** → Save → **Default global**.

5.3 Floor Planning & Power Planning

- **Floorplan Parameters:** Governed by Aspect Ratio (W/H), core utilization, and channel spacing between the boundary and I/O pins.

Additional technical note:

The notes write the aspect ratio parameter as W/H [VERIFY: handwritten note may say W/W or W/H, standard is Width/Height].

- **Specify Floor Plan:** Go to **Floorplan** → **Specify Floor Plan** → Set Ratio to **1** → Core to Die Boundary to **All 6** → **OK**
- **Global Net Connections:**
 - **Power** → **Connect Global net**
 - Pin Name & To Global Net → **VDD** → **Add to list**
 - Pin Name & To Global Net → **VSS** → **Add to list** → **OK**
- **Add Power Ring:**
 - **Power** → **Power Planning** → **Add Ring** → Nets: **VDD**, **VSS**

- Configuration (For 90nm PDK): Left/Right = Metal 8 (low resistivity), Top/Bottom = Metal 9
- Tick offset, set Width to **1 (All)** → **update**

• **Add Power Stripes:**

- **Power** → **Power Planning** → **Add Stripe**
- Nets: **VDD**, **VSS**. Layer: **M-9**, Width (**w**): **1**, Spacing (**s**): **0.4**
- Set Pattern: No. of sets = **3** → **OK**
- Repeat process for Metal 8.

Additional technical note:

The handwritten text says "Adding power strings," which is a common miswriting of "power stripes".

- **Special Routing: Route** → **Special Route** → Nets: **VDD**, **VSS** → **OK**

5.4 Placement

• **Physical Cells:**

- **Place** → **Physical Cell** → **Add End Cap** → Set Pre cap to **FILL 1** (left) and Post cap to **FILL 2** (Right) → **OK**
- **Place** → **Physical Cell** → **Add Well Tap** → Cell name = **FILL 4**, Dist interval = **32**, Tick **fixed** → **OK**

• **Standard Cells & I/Os:**

- **Place** → **Place Standard Cell** → Mode → Tick **place I/O pins** → **OK**

Tip:

Uncheck the layer box to visibly see the standard cell placement and the yellow box pins inside the cells.

5.5 Timing, CTS, and Optimization

- **Pre-CTS Timing Check: Timing** → **Report timing** → **Pre-CTS** → **Setup**
- **Design Optimization: ECO** → **Optimize Design** → **OK**
- **Clock Tree Synthesis (CTS): Clock** → **Clock Tree Debugger** [VERIFY: Standard process refers to Clock Tree Synthesis] → **OK** → **Yes**
- **Post-CTS Timing Check: Timing** → **Report timing** → **Post-CTS** → **Setup** (observe the number of paths diff/clock), then run **Post-CTS** → **Hold**
- **Analysis Mode Adjustment: Tools** → **Set Mode** → **Specify Analysis mode** → select **on-chip variation** → Tick **CPPR** → **OK**

Additional technical note:

CPPR stands for Clock Path Pessimism Removal.

5.6 Routing and ECO Debugging

- **Main Routing:** **Route** → **Nano Route** → **Route** → Tick **Timing Driven** and **SI Driven** → **OK**
- **Timing Closure / ECO:**

- To resolve slack time or timing violations, go to **Timing** → **Debug timing** → **OK**
- Select any violating signal → Right-click → **show Timing path Analyzer**
- Select the clock with the highest time/delay → Right-click → **Interactive ECO/what if** → **change cell**
- **For Hold Violations:** Repeat the same steps, but select **Add Repeater** (e.g., using buffer **BUFX12**)

5.7 Final Export Generation

- **Save Innovus Database:** **File** → **Save** → **Innovus** → **vendingMachine.enc**
- **Save GDSII:** **File** → **Save** → **GDS** → output file = **vm.gds** , map file = **StreamOut.map**

6. External Learning References

- **abhiyanta.com** → RTL DV, SV (SystemVerilog), Syn STA (Synthesis & Static Timing Analysis), PD (Physical Design), UVM (Universal Verification Methodology)
- Cadence → Learning Maps